Columbia University - June 2025

A data breach affected 870,000 people.

Two months undetected.

The attacker started with a regular account.

Then escalated.

NoEsc

A Comparative Analysis of Rule-Based and Machine Learning-Based Detection of Privilege Escalation in Linux Systems

CMSC 190 Special Problem

Klenn Jakek V. Borja

Two approaches exist for detection.
Neither is a clear winner.

Rules-Based	Machine Learning
☑ Transparent	☑ Broad Coverage
☑ Zero false positives	☑ Catches novel attacks
☒ Rigid	☒ Alert Fatigue
☒ Blind to novel attacks	☒ Black box

Which one actually works in an academic Linux environment?
That gap is what this study addresses.

Objectives

Introduction

Objectives

1.) Design and implement a heuristic rule-based engine for transparent privilege escalation detection

2.) Design and implement a machine learning-based engine, including feature engineering for raw audit logs

3.) Evaluate the effectiveness and performance overhead of both engines against a standardized set of attacks

4.) Analyze the trade-offs between the two approaches regarding detection accuracy, performance, and interpretability

Methodology

NoEsc Architecture

NoEsc Architecture

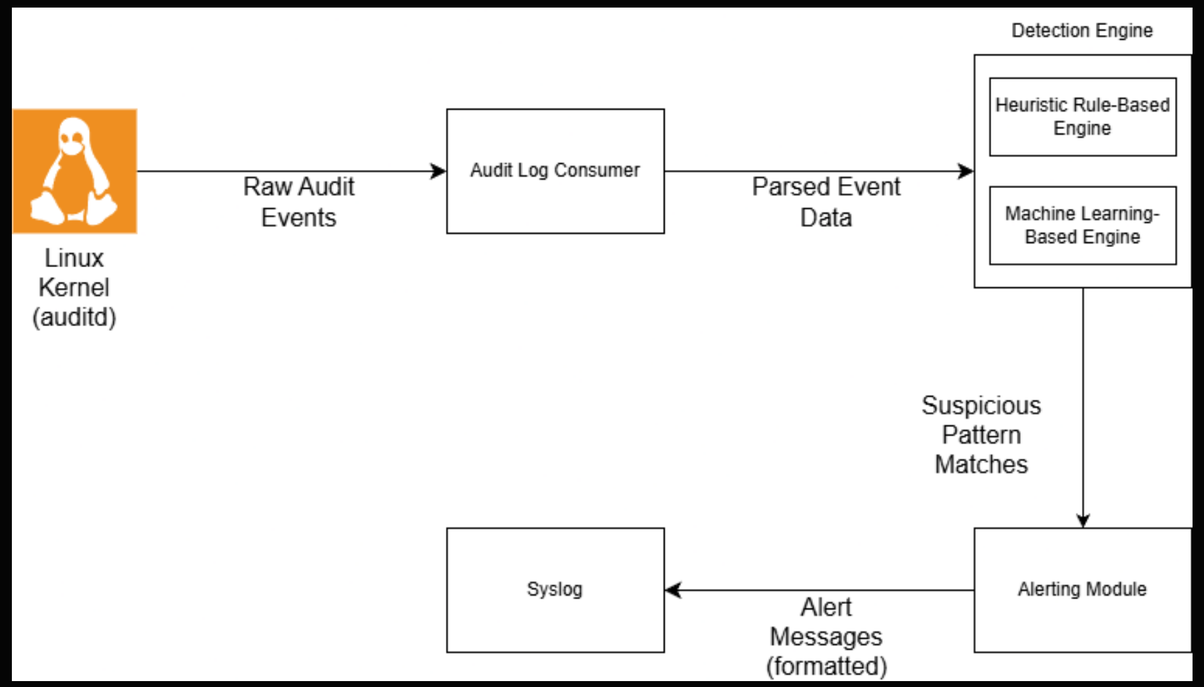

Fig 1: System Architecture of the NoEsc daemon

Key decisions:

- auditd via audispd plugin – real-time, no file polling
- C++ core daemon – lightweight, low overhead
- Non-blocking UDS bridge – ML never slows rule engine

Heuristic Rule-Based Engine

Rule-Based Engine

Deterministic
Stateful
Interpretable

Heuristic Rule-Based Engine

Rule-Based Engine

Deterministic • Stateful • Interpretable

3 Detection Vectors

SUID/SGID Abuse

Path whitelist + euid check

Sudo Misuse

Stateful scoring system

auth fail ▷ +1

High risk command ▷ +1

WARNING @	CRITICAL @
15 points	20 points

Sensitive File Tampering

File-watch on:

- ▷ /etc/passwd
- ▷ /etc/shadow
- ▷ /etc/sudoers

Heuristic Rule-Based Engine

Rule-Based Engine

```
type=SYSCALL msg=audit(1749823200.341:1042):
arch=c000003e syscall=59 success=yes
exe="/tmp/sh" auid=1001 uid=1001
euid=0 egid=0 pid=4821
key=“benign_exec"
```

Heuristic Rule-Based Engine

Rule-Based Engine

```
type=SYSCALL exe="/tmp/sh" auid=1001 euid=0
```

Rule-Based Engine

type=SYSCALL exe="/tmp/sh" auid=1001 euid=0

Rule sees:

~~non-root user~~

~~euid = 0 (root)~~

~~path in /tmp~~

CRITICAL ALERT!

Machine Learning-Based Engine

Machine Learning Engine

Data-driven
Behavioral
Generalizable

Machine Learning Engine

Data-driven • Behavioral • Generalizable

Feature Engineering Pipeline

Raw log

```
type=SYSCALL msg=audit(1749823200.341:1042) syscall=59
exe="/tmp/sh" auid=1001 euid=0 key="weird_exec"
```

▽

Group by PIDs

```
open -> read -> mmap -> execve -> …
```

▽

Generate (2,3)-grams + TF-IDF

```
[open→read] [read→mmap] [mmap→execve]
rare suspicious sequences weighted higher
```

▽

Append contextual features

```
ctx_exe_in_tmp = 1
ctx_euid_is_root = 1
ctx_auid_euid_mismatch = 1
…
```

Machine Learning-Based Engine

SVM Classification

MALICIOUS!

Machine Learning Engine

Data-driven • Behavioral • Generalizable

Feature Engineering Pipeline

Raw auditd logs

```
…
type=SYSCALL msg=audit(1749823200.341:1042): arch=c000003e syscall=257 success=yes exe="/tmp/sh"
auid=1001 euid=1001 pid=4359
type=SYSCALL msg=audit(1749823200.342:1043): arch=c000003e syscall=3 success=yes
exe="/usr/bin/cat" auid=1001 euid=1001 pid=1200
type=SYSCALL msg=audit(1749823200.345:1044): arch=c000003e syscall=0 success=yes exe="/tmp/sh"
auid=1001 euid=1001 pid=4359
type=SYSCALL msg=audit(1749823200.348:1045): arch=c000003e syscall=59 success=yes exe="/tmp/sh"
auid=1001 euid=0 pid=4359
…
```

Machine Learning Engine

Data-driven • Behavioral • Generalizable

Feature Engineering Pipeline

Raw auditd logs

```
…
type=SYSCALL msg=audit(1749823200.341:1042): arch=c000003e syscall=257 success=yes exe="/tmp/sh"
auid=1001 euid=1001 pid=4359
type=SYSCALL msg=audit(1749823200.342:1043): arch=c000003e syscall=3 success=yes
exe="/usr/bin/cat" auid=1001 euid=1001 pid=1200
type=SYSCALL msg=audit(1749823200.345:1044): arch=c000003e syscall=0 success=yes exe="/tmp/sh"
auid=1001 euid=1001 pid=4359
type=SYSCALL msg=audit(1749823200.348:1045): arch=c000003e syscall=59 success=yes exe="/tmp/sh"
auid=1001 euid=0 pid=4359
…
```

Machine Learning Engine

Data-driven • Behavioral • Generalizable

Feature Engineering Pipeline

Raw auditd logs

```
…
type=SYSCALL msg=audit(1749823200.341:1042): arch=c000003e syscall=257 success=yes exe="/tmp/sh"
auid=1001 euid=1001 pid=4359
type=SYSCALL msg=audit(1749823200.345:1044): arch=c000003e syscall=0 success=yes exe="/tmp/sh"
auid=1001 euid=1001 pid=4359
type=SYSCALL msg=audit(1749823200.348:1045): arch=c000003e syscall=59 success=yes exe="/tmp/sh"
auid=1001 euid=0 pid=4359
…
```

Machine Learning Engine

Data-driven • Behavioral • Generalizable

Feature Engineering Pipeline

Group by PIDs

pid = 4359 [257 -> 0 -> 59]

Machine Learning Engine

Data-driven • Behavioral • Generalizable

Feature Engineering Pipeline

Group by PIDs

pid = 4359 [openat -> read -> execve]

Machine Learning Engine

Data-driven • Behavioral • Generalizable

Feature Engineering Pipeline

(2,3)-grams + TF-IDF Weighting

2-gram

Chunk 1: [openat -> read] = 0.05 (common, low weight)

Chunk 2: [read -> execve] = 0.92 (rare/sus, high weight)

Vector array: [0.00, 0.05, 0.00, ..., 0.92, 0.00]

Machine Learning Engine

Data-driven • Behavioral • Generalizable

Feature Engineering Pipeline

Contextual Features

Vector array: [0.00, 0.05, 0.00, ..., 0.92, 0.00] + Context

Context:

```
[ctx_exe_in_tmp]       = 1  (It ran from /tmp)
[ctx_euid_is_root]     = 1  (It successfully got root)
[ctx_auid_mismatch]    = 1  (The user's ID doesn't match root)
```

Machine Learning Engine

Data-driven • Behavioral • Generalizable

Feature Engineering Pipeline

SVM CLASSIFIER → MALICIOUS!

Machine Learning Engine

Data-driven • Behavioral • Generalizable

Dual Model Architecture

sequence length = 5

▽

≥ 3 events ▷ Long Model (v4)[primary]
F1 = 0.998

1–2 events ▷ Short Model (v1) [companion]
F1 = 0.785
+ Score demotion

Dataset Construction

Benign Data

1.2+ Million raw kernel logs collected from live ICS lab workstations

Malicious Data

Generated via attack script scenarios

Data Curation & Compression

Filter irrelevant logs

Group raw logs by PIDs

Final Training Volume

123,082 total valid process sequences

Training: 86,157

Testing: 36,925

Results and Discussion

NoEsc

SVM Model Comparison

TABLE II

SVM MODEL PERFORMANCE COMPARISON

Model	min	Acc	Prec	Rec	F1	MCC
v1	2	.968	.971	.991	.981	.894
v3	3	.952	.906	1.00	.951	.908
v4 (min2)	2	.795	.999	.765	.867	.543
v4 (min3)	**3**	**.998**	**.998**	**.999**	**.998**	**.995**
short_v1	1	.821	.960	.663	.785	.672

← BEST

Most impactful hyperparameter: min_events
Recall: 0.765 → 0.999 just by filtering sequences shorter than 3 events

Rule-Based Engine Results

TABLE III

RULE-BASED ENGINE DETECTION RESULTS

Vector	Scenario	Exp.	Res.
SUID	`/tmp/backdoor`	CRIT	TP ✓
SUID	`/home/.../myshell`	WARN	TP ✓
SUID	`/opt/sandbox/`	None	TN ✓
Sudo	4 cmds + auth fail	W+C	TP ✓
File	Non-root → `shadow`	CRIT	TP ✓
File	Root → `shadow`	None	TN ✓
Benign	`sudo apt update`	None	TN ✓

100% TPR — 0% FPR

Fairness Comparison

TABLE IV

FAIRNESS COMPARISON: ALERT VOLUME (3-RUN AVG.)

Metric	ML	Rules
Mean alerts	108.00	7.00
Alert rate (%)	0.119	0.008
Per-run range	88–121	7/7/7
WARNING	—	2.00
CRITICAL	—	5.00

ML generates ~15× more alerts

Why 108 vs. 7?

Metric	ML	Rules
Mean alerts	108.00	7.00

Metric	ML	Rules
Mean alerts	108.00	7.00

Why 108 vs. 7?

This is not a model failure.

It is a fundamental
architectural difference.

Metric	ML	Rules
Mean alerts	108.00	7.00

Why 108 vs. 7?

This is not a model failure. It is a fundamental architectural difference.

Heuristic Rule-Based Engine

evaluates three narrow hardcoded patterns

Machine Learning-Based Engine

classifies every single process on the system

Why 108 vs. 7?

Why 108 vs. 7?

Why 28 vs. 7?

28 vs. 7

Solution: Process Aware Whitelist

```
# --- System package managers ---
/usr/bin/snap
/usr/lib/snapd/snap-update-ns
/usr/lib/snapd/snap-confine
/usr/lib/snapd/snapd
/usr/lib/snapd/snapctl
/snap/
```

```
# --- Scheduled tasks ---
/usr/sbin/cron
/usr/sbin/atd
/usr/sbin/anacron
```

Known noisy daemons suppressed before inference

Before whitelist: 108 alerts

After whitelist: 28 alerts

74% reduction

0 true positives lost

15:1 -> 4:1

Performance Overhead

Macro Benchmark

TABLE VI

MACRO BENCHMARK: BUILD TIME OVERHEAD (5-RUN AVG.)

Condition	Avg. Build Time (s)	Overhead
Baseline (no audit rules)	3.5550	—
With NoEsc rules active	3.5556	+0.02%

Audit rule overhead:
+0.02% (negligible)

Micro Benchmark

ML Throughput: 40,393 events/sec

Real lab rate: < 1 event/sec

Massive headroom.

Verdict

Heuristic Rule-Based Engine

= Precision

Machine Learning-Based Engine

= Coverage

Verdict

Heuristic Rule-Based Engine

Machine Learning-Based Engine

Neither alone is enough.

Hybrid is what works.